NODE.JS BACKEND DEVELOPMENT

Lecture 6: MongoDB, CRUD, Postman & Mongoose

Detailed English Notes Based on the Provided Lecture Transcript

What this lecture covers

• Why applications need databases and how client, application server, and database work together.

• MongoDB installation and starting the MongoDB server/shell.

• SQL terminology compared with MongoDB terminology.

• MongoDB database, collection, document, field, and automatic ObjectId.

• Creating, reading, updating, and deleting documents using MongoDB shell commands.

• MongoDB Compass as a graphical interface for viewing and editing data.

• How API requests and JSON bodies are tested with Postman.

• Why a database driver is needed, and why the lecture introduces Mongoose (an ODM).

> Important: Commands and examples below are written in a clean, readable form. Some command spellings in the speech-to-text transcript are inaccurate, so the notes use the standard MongoDB syntax where the intended command is clear.

1. Learning approach: understand one layer at a time

The instructor begins by advising learners not to carry every old doubt into a new topic. First understand what the current lesson is explaining, then identify the exact point that remains unclear. For backend development, this means learning the request flow and the role of each component before memorising commands.

Recommended learning method

1. Read one concept and explain it in your own words.

2. Run the corresponding command yourself instead of only watching the instructor.

3. Check the result and compare it with the expected output.

4. If an error appears, read the full message and investigate the exact cause.

5. Move forward only after you can repeat the operation without copying blindly.

2. Why a database is needed

A database stores information that an application must remember and retrieve later. A website may need to store users, email addresses, products, menu items, orders, employee details, salaries, ingredients, sales counts, or other records. Without persistent storage, the server would have no reliable place to keep this application data between requests.

Restaurant example

Imagine a restaurant website. The application may need to know the menu, prices, ingredients, whether an item is a drink, the number of units sold, and details about employees. A server can handle simple responses itself, but if it needs stored menu or employee data, it asks the database for that information.

Three main parts

• **Client:** the browser or frontend application that sends a request and displays the response.

• **Application server:** the backend (in this course, Node.js with Express) that receives requests, runs application logic, and decides whether database access is required.

• **Database server:** the database system that stores and retrieves persistent data.

> Typical flow: Client → Node.js/Express server → MongoDB server → Node.js/Express server → Client.

The application server and database server are separate roles. MongoDB provides its own server process; the developer generally installs/configures and runs it rather than writing a database server from scratch. Both servers must be reachable for database-backed features to work.

3. Choosing a database

The lecture mentions several database options, including SQL systems such as PostgreSQL and non-relational/document databases such as MongoDB. The right choice depends on the application's needs; this lecture chooses MongoDB so learners can practise its commands and later connect it to Node.js.

SQL and NoSQL at a glance

Relational / SQL terminology	MongoDB terminology
Database	Database

Relational / SQL terminology	MongoDB terminology
Table	Collection
Row / record	Document
Column	Field
Schema usually defined around tables/columns	Documents are BSON-like objects; structure can vary, though an application may

The underlying ideas are similar: organise data, insert records, retrieve records, update them, and delete them. The syntax and data model differ.

4. MongoDB installation and local server

The lecture demonstrates a local MongoDB setup. MongoDB Community Server is the database server software. After installation, start the server using the method appropriate for your operating system. On many systems MongoDB runs as a service; on others, the setup may require a command or service manager. Exact steps vary by OS, MongoDB version, and installation method.

> Do not assume every operating system uses the same start command. Follow the official installation guide for your platform and verify that the service is running.

MongoDB server vs. mongosh

• **MongoDB server (mongod):** the database process that listens for connections and manages data.

• **mongosh:** the MongoDB shell, an interactive command-line client used to connect to the server and run database commands.

• **MongoDB Compass:** a graphical user interface for browsing and editing databases and documents.

Basic checks

```
mongosh --version
mongosh
show dbs
```

The version command checks whether the shell is installed and available on PATH. Running `mongosh` attempts to connect to a local MongoDB server using its default connection settings. If connection fails, verify that the server is installed, running, and listening at the expected address and port.

The transcript shows a service-start command on the instructor's machine. Treat that command as platform-specific rather than universal. On Windows, macOS, and Linux the correct method can differ.

5. MongoDB's core data structure

Database

A database is a named container that holds related collections. The lecture uses a database called `db` in examples, but you may choose a meaningful name such as `restaurantDB` or `appDB`.

Collection

A collection is broadly comparable to a SQL table. For example, a restaurant application might have `users`, `products`, and `feedback` collections.

Document

A document is one stored record inside a collection. MongoDB documents use a JSON-like structure, stored internally as BSON. A user document could contain a name, age, username, and email.

Field

A field is one named value inside a document, such as `name`, `age`, or `email`. A field may contain a string, number, Boolean, array, nested object, date, or other supported BSON type.

The automatic _id field

MongoDB assigns an `_id` value to a document when one is not supplied. It is commonly an ObjectId and uniquely identifies that document within its collection. The application can use it to find a specific record.

```
{
  "_id": ObjectId("..."),
  "name": "Prince",
  "age": 23,
  "email": "prince@example.com",
  "address": {
    "city": "Delhi",
    "state": "Delhi"
  }
}
```

Notice that `address` is a nested object. MongoDB documents can represent structured data without flattening every value into a separate column.

6. Basic MongoDB shell commands

Select or switch to a database

```
use appDB
show dbs
show collections
```

`use appDB` switches the shell's current database context. MongoDB may not show a newly selected database in `show dbs` until data has been stored in it.

Create a collection

```
db.createCollection("users")
show collections
```

MongoDB can also create a collection implicitly when you first insert a document into it. The explicit command is useful when you want to create a collection in advance.

Insert one document — Create

```
db.users.insertOne({
  username: "prince",
  age: 23
})
```

If successful, the result reports an acknowledgement and the inserted document's identifier. MongoDB creates `_id` automatically if you did not provide one.

Read documents

```
db.users.find()
db.users.find({ username: "prince" })
db.users.find({ age: { $gt: 21 } })
db.users.find({ age: { $lt: 24 } })
```

`find()` retrieves matching documents. An empty filter matches all documents in the collection. Operators such as `$gt` mean “greater than” and `$lt` mean “less than.”

Update a document

```
db.users.updateOne(
  { username: "prince" },
  { $set: { age: 27 } }
)
```

The first argument identifies which document to match. The second argument describes the update. `$set` changes the specified field without replacing the entire document. Check the update result and query the record again to verify it.

Delete a document

```
db.users.deleteOne({ username: "prince" })
```

The filter identifies the document to delete. Be careful with filters: a mistaken filter can affect the wrong data. In a practice database, check the target with `find()` before deleting.

> CRUD = Create, Read, Update, Delete. These four operations are the foundation of working with stored records.

7. MongoDB Compass

MongoDB Compass is a graphical user interface (GUI) for working with MongoDB. It lets you inspect databases, collections, and documents visually rather than relying only on a terminal. The lecture also mentions Robo 3T as an alternative GUI, while focusing on Compass.

Typical Compass workflow

1. Start the MongoDB server.

2. Open Compass and connect to the local server using the connection string appropriate to your installation, commonly `mongodb://127.0.0.1:27017` for a default local setup.

3. Open the required database and collection.

4. View documents and use the interface to insert, edit, or delete a document.

5. Return to the shell and query the same collection to verify the change.

Compass is a visual client; it does not replace the database server. If the server is stopped or the connection settings are wrong, Compass cannot access the data.

8. Designing data before writing code

The lecture uses forms as an example of data design. A frontend form asks for values such as name, email, address, or product details. When the user submits it, the frontend packages the values into a structured request and sends them to an API endpoint. The backend receives and validates the data, applies business logic, and then stores it in the database.

Example product document

```
{
  "name": "Mango Smoothie",
  "price": 4.99,
  "taste": "sweet",
  "isDrink": true,
  "ingredients": ["mango", "milk", "ice"],
  "unitsSold": 45
}
```

The fields depend on the application's needs. A user document and a product document will not necessarily contain the same fields. Before implementing a form or API, decide what information is needed, what types each value should have, and which fields are required.

Frontend form vs. API payload

• The form is the visible interface used to collect data.

• The payload is the structured data sent to the backend, often JSON.

• The API endpoint and HTTP method define where and how the request is sent.

• The backend must validate the incoming payload rather than trusting every value supplied by the client.

9. HTTP methods and Postman

GET vs POST

Method	Typical purpose	Request body
GET	Retrieve or read information	Usually not used for application data payloads
POST	Submit data; often create a resource or trigger an operation	Commonly contains JSON data

A route defined only for GET will not automatically accept POST. If you send a POST request to an endpoint that has no POST handler, Express normally returns a “Cannot POST …” response. The route must be implemented for the method and path you are calling.

Testing JSON in Postman

1. Start the Node.js/Express server.

2. Create a request in Postman and enter the URL, such as `http://localhost:3000/person`.

3. Choose POST if the endpoint is intended to accept new data.

4. Open Body → raw and select JSON.

5. Paste a valid JSON object, for example the person payload below.

6. Click Send and inspect the status code and response body.

```
{
  "name": "Alex",
  "age": 28,
  "mobile": "9876543210",
  "email": "alex@example.com",
  "address": "Delhi",
  "salary": 50000
}
```

In Express, a common setup is `app.use(express.json())` so JSON request bodies can be parsed into `req.body`. Without appropriate body parsing, the backend may not read the submitted JSON as expected.

```
const express = require("express");
const app = express();

app.use(express.json());

app.get("/", (req, res) => {
  res.send("Welcome!");
});

app.post("/person", (req, res) => {
```

```
  const personData = req.body;
  console.log(personData);

  res.status(201).json({
    message: "Request received",
    data: personData
  });
});

app.listen(3000, () => {
  console.log("Server running on port 3000");
});
```

This example only receives and returns the data. It does not save anything to MongoDB. Database persistence requires the connection and database operation discussed next.

10. How Node.js communicates with MongoDB

Node.js and MongoDB do not simply share the same internal language or process. The Node.js application uses a client library to communicate with the MongoDB server over the MongoDB wire protocol. This library is called a MongoDB driver.

Native MongoDB Node.js driver

The official MongoDB Node.js driver provides APIs for connecting to the database and running operations such as insert, find, update, and delete. It handles protocol communication and exposes JavaScript methods that the application can call.

Mongoose: an ODM layer

Mongoose is an Object Data Modeling (ODM) library commonly used with Node.js and MongoDB. It builds on the underlying driver and offers additional application-level structure, including schemas, models, validation, middleware, and convenient query APIs.

Conceptual stack: Node.js/Express application → Mongoose → MongoDB Node.js driver → MongoDB server.

Why use Mongoose?

• **Schemas:** define the expected shape of a document at the application layer.

• **Models:** provide a model-based interface for creating and querying documents in a collection.

• **Validation:** check values against rules before saving, such as requiring a name or validating an email format.

• **Consistent structure:** help prevent accidental, unexpected fields and keep application data organised.

• **Convenient operations:** make common create/read/update/delete workflows easier to express.

• **Middleware and hooks:** allow logic to run at certain stages of a model operation when needed.

MongoDB itself is flexible about document structure. A Mongoose schema is an application-level contract; it does not mean MongoDB has become a relational SQL database. Validation and schema behavior depend on how the application and model are configured.

Example schema and model

```
const mongoose = require("mongoose");

const personSchema = new mongoose.Schema({
  name: {
    type: String,
    required: true
  },
  age: {
    type: Number,
```

```
    min: 0
  },
  email: {
    type: String,
    required: true,
    lowercase: true,
    trim: true
  }
});

const Person = mongoose.model("Person", personSchema);

module.exports = Person;
```

The schema is the blueprint for the application's Person documents. The model is the interface used by application code to create and query those documents. This is an introductory example; a real application should also consider email validation, uniqueness requirements, error handling, and security.

Native driver vs. Mongoose

Native driver	Mongoose
Official low-level Node.js interface to MongoDB	ODM library built on top of the driver
More direct control; developer handles more structure and validation	Adds schemas, models, validation, middleware, and convenience APIs
Useful when you want the driver's direct API	Useful when a model-driven application benefits from defined data shapes

The lecture recommends learning Mongoose because it reduces repetitive work for many common application patterns. However, the native driver is still a valid tool; the best choice depends on project requirements and developer preference.

11. Full request flow: from form to database

1. A user fills out a frontend form.
2. The frontend converts the values into a JSON payload.
3. The frontend sends an HTTP request to a defined Express endpoint.
4. Express parses the request and validates its data.
5. The route calls the relevant model or database layer.
6. Mongoose applies the configured schema/validation and communicates through the MongoDB driver.
7. MongoDB processes the operation and returns a result.
8. The backend sends an appropriate HTTP response to the client.
9. The frontend displays a success or error message.

> A successful HTTP response alone does not prove that data was stored. Confirm that the route actually performs a database operation, then verify the document in Compass or with a shell query.

12. Common mistakes and troubleshooting

• **MongoDB connection refused:** check whether the database server is running and whether the connection URI, host, and port are correct.

• **mongosh command not found:** verify that MongoDB Shell is installed and available on PATH.

• **Database does not appear in show dbs:** a newly selected database may not appear until data has been written to it.

• **Collection is missing:** check the current database using `db` and list collections with `show collections`.

• **Cannot POST /path:** ensure the Express app has a POST route for the exact path and that the request uses POST.

• **req.body is undefined or empty:** verify `app.use(express.json())`, Postman's raw JSON mode, and the request's content type.

• **Update changes nothing:** check the filter first and inspect `matchedCount` and `modifiedCount` where available.

• **Delete removes the wrong record:** inspect the filter and query the target before executing the delete.

• **Data is returned but not saved:** check whether the route actually calls an insert/save operation; echoing `req.body` is not persistence.

• **Compass cannot connect:** ensure the server is running and use the correct local connection URI.

13. Hands-on practice checklist

■ Confirm MongoDB Server and mongosh are installed.

■ Start the database server and connect with mongosh.

■ Create or switch to a practice database named appDB.

■ Create a users collection or let MongoDB create it on first insert.

■ Insert two documents with different names and ages.

■ Retrieve all users, then filter by username.

■ Use $gt and $lt to query numeric fields.

■ Update one document with $set and verify the result.

■ Delete one practice document and confirm the remaining records.

■ Connect MongoDB Compass to the local server and inspect the same collection.

■ Start a simple Express server and test a GET route in Postman.

■ Add a POST route, send a JSON body, and inspect req.body.

■ Explain why the POST route above does not persist data until it calls MongoDB.

■ Write down how the MongoDB driver differs from Mongoose.

14. Quick revision sheet

Term	Meaning
Database	Stores application data persistently.
Client	Sends requests and displays responses.
Application server	Handles HTTP requests, application logic, and database calls.
MongoDB server	Stores and retrieves MongoDB data.
mongosh	Interactive command-line shell for MongoDB.

Term	Meaning
Compass	GUI to inspect and manage MongoDB data.
Collection	Group of MongoDB documents; broadly like a SQL table.
Document	One record represented as a JSON-like object.
Field	A named value in a document.
_id	Document identifier assigned automatically if omitted.
CRUD	Create, Read, Update, Delete.
GET	Usually used to retrieve data.
POST	Often used to submit data or create a resource.
Postman	Tool for sending HTTP requests and testing APIs.
MongoDB driver	Node.js library that communicates with the MongoDB server.
Mongoose	ODM that adds schemas, models, validation, and convenience APIs.

15. Self-check questions

1. Why does a web application need a database?
2. What is the difference between an application server and a database server?
3. Map SQL table, row, and column to MongoDB terminology.
4. What is the purpose of the MongoDB _id field?
5. Write commands to insert, find, update, and delete a user document.
6. What is the difference between GET and POST in the examples from this lecture?
7. Why can Postman be useful before a frontend form has been built?
8. Why is receiving a request body not the same as saving it to MongoDB?
9. What role does the MongoDB Node.js driver play?
10. What extra application-level features does Mongoose provide?
11. What should you check first if Compass cannot connect to a local database?

16. Key takeaways

• Backend applications commonly use a client, an application server, and a database server.

• MongoDB stores data as documents grouped into collections inside databases.

• CRUD operations are the everyday foundation of database work.

• Postman can simulate frontend requests and is useful for testing GET and POST routes.

• A route must be implemented for the HTTP method and URL being requested.

• A request is not persisted unless backend code actually performs a database operation.

• The MongoDB driver connects Node.js to MongoDB; Mongoose adds a structured model and validation layer.

• Practice each command and verify the result instead of only memorising syntax.

End of Lecture 6 notes. These notes follow the supplied transcript's progression from database fundamentals through CRUD, Postman, and the introduction to Mongoose. The transcript previews Mongoose but does not fully teach a complete production database integration, so this document treats the integration code as an introductory concept.